

算法分析与设计

Chapter 1 概述 (w1)

一. 算法:

1. 定义: 解决问题的一种方法或一个过程。

2. 性质: 是有穷序列, 针对若干指令, 满足:

(1) 输入: 有外部提供的 0/多个输入。

(2) 输出: 产生 1/多个输出

(3) 确定性: 组成算法的每条指令清晰、无歧义。

(4) 有限性: 算法中每条指令执行次数有限, 执行每条指令的时间也有限。

3. 案例: 求 $\gcd(m, n)$ 使用欧几里德算法 (递归) $\gcd(m, n) = \gcd(n, m \bmod n)$

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

(用定义从较小值开始试, 效率低)

(质因数分解法是不满足确定性!) 满足

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$
 $n \leftarrow r$

return m

二. 程序:

1. 定义: 算法用某种程序语言的具体实现。

2. 程序可以不满足有限性 (4), 如 OS 由于是无限循环的程序, 因而 OS 是程序但不是算法。

3. 算法是问题的程序化解决方案, 证明正确性用数学归纳法; 证明不正确只需一个反例。

三. 算法复杂性分析: 计算算法所需的计算机资源

1. 时间复杂性 $T(n)$

2. 空间复杂性 $S(n)$ (由于硬件资源的发展),

其中 n 是问题的规模 (输入大小)。

3. 分析方式: 事后统计/事前分析

时间复杂度: 每条语句的频度之和, 分类: 最坏, 最好, 平均, 计算机关, 最坏复杂度

eg: 分析顺序查找 $T_{\max}(n) = O(n)$ $T_{\min}(n) = O(1)$, $T_{avg}(n) = \sum_{i=1}^n p(i) T(i)$

$$= (1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n}) + n(1-p)$$

$$= \frac{p}{2}(n+1) + n(1-p)$$

4. 算法渐近复杂性: 不用精确计算, $t(n)$ 是 $T(n)$ 的渐近表达式

渐近分析记号 $\Theta, O, \Omega, o, \omega$, 以下设 u_n 有 $f(n) \geq 0$ 且 $g(n) \geq 0$

(1) 渐近上界记号 O eg: $2n+3 = O(n^2)$ $3n^2 = O(n^4)$

$$O(g(n)) = \{f(n) \mid \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0 \text{ 有 } 0 \leq f(n) \leq cg(n)\}$$

$O(g(n))$ 表示增长次数 $\leq g(n)$ 的函数集合

(2) 渐近下界记号 Ω eg: $3n^2 = \Omega(n^2)$, $n^2 = \Omega(n^2)$

$$\Omega(g(n)) = \{f(n) \mid \exists c > 0, n_0 > 0 \text{ s.t. } \forall n \geq n_0 \text{ 有 } 0 \leq cg(n) \leq f(n)\}$$

$\Omega(g(n))$ 表示增长次数 $\geq g(n)$ 的函数集合

(3) 渐近紧界记号 Θ $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

$$\Theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0 > 0 \text{ s.t. } \forall n \geq n_0 \text{ 有 } c_1 g(n) \leq f(n) \leq c_2 g(n)\}$$

$\Theta(g(n))$ 表示增长次数 $= g(n)$ 的函数集合

Big-O 略去低阶、常数系数项

eg: 插入排序 $O(n^2)$, 归并排序 $\Omega(n)$, 合并 $O(n \log n)$

(4) 非紧上界记号 o $0 \leq f(n) < cg(n)$ $\forall c > 0, \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0, 0 \leq f(n) < cg(n)$

(5) 非紧下界记号 ω $0 \leq cg(n) < f(n)$

$$\Leftrightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

5. 渐近记号的性质:

(1) 传递性: $f(n) = A(g(n)), g(n) = A(h(n)) \Rightarrow f(n) = A(h(n))$, A 为 $\Theta/O/\Omega/o/\omega$

(2) 反身性: $f(n) = A(f(n))$, A 为 $\Theta/O/\Omega/o/\omega$

(3) 对称性: $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

(4) 互对称性: $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$; $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$

(5) 算术运算: $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\}) = O(f(n) + g(n))$
 $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$ $\forall f(n) \in O(f), f(n) \in O(g(n))$
 $O(cf(n)) = O(f(n))$ $f(n) + f(n) =$
 $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$ $f(n) \in O(f), f(n) \in O(f)$

★ 期末证明: $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$

pf: $\forall f_1(n) \in O(f(n)), \exists C_1, n_1 \in \mathbb{N}$ s.t. $\forall n > n_1, f_1(n) \leq C_1 f(n)$

$\forall g_1(n) \in O(g(n)), \exists C_2, n_2 \in \mathbb{N}$ s.t. $\forall n > n_2, g_1(n) \leq C_2 g(n)$

令 $C_3 = \max\{C_1, C_2\}, n_3 = \max\{n_1, n_2\}, h(n) = \max\{f(n), g(n)\}$

则 $\forall n > n_3, f_1(n) + g_1(n) \leq C_1 f(n) + C_2 g(n) \leq C_3 f(n) + C_3 g(n) = C_3 (f(n) + g(n))$

$\Rightarrow$ ~~$f_1(n) + g_1(n) = O(\max\{f(n), g(n)\})$~~ $\leq C_3 \cdot 2h(n)$

复杂性大小关系: $O(1) < O(\log n) < O(\log^2 n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

分析代码方式: eg: 插入排序 $O(n^2)$

C++ 操作 $\rightarrow$ 常数时间

顺序语句 $\rightarrow$ 语句时间和

条件语句 $\rightarrow$ 较大语句分支 + 条件测试

循环 $\rightarrow$ 迭代和

函数 $\rightarrow$ 函数体代价

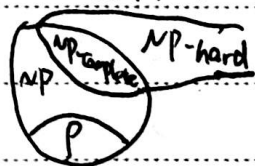
递归 $\rightarrow$ 求解递归方程, 依赖于子问题是几个数、规模、集合时间

最优算法, 排序问题最优 $O(n \log n)$, 如堆排序可实现 插入排序 已排序问题复杂性 $O(n)$
 倒序问题为 $O(n^2)$

6. NP 完全性理论 $P = NP?$

P 类问题: 所有可在多项式时间内求解的判定问题, eg: $O(n^2), O(n^4), O(n \log n)$

NP 类问题: 所有的非确定性多项式时间可解的判定问题, 可能恰好有解



PP, ZPP, PL, BPP, RP, EXP, NEXP, ...

确定和非确定的图灵机是否具有同样的功能

能用多项式时间验证解的问题是能否在多项式时间内找到解

★ 7. 主定理 (Master Theorem) 用于求解递归式算法的时间复杂度

Thm. 设 $a \geq 1$, $b \geq 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负函数且 $T(n) = aT(\frac{n}{b}) + f(n)$

则: (1) 子问题主导 若 $\epsilon > 0$, 有 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $T(n) = O(n^{\log_b a})$

即 $f(n)$ 的增长慢于 $n^{\log_b a}$ (多项式意义上)

(2) 平衡子 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$, 则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
 即 $f(n)$ 与 $n^{\log_b a}$ 同阶 (故乘以 $\log$ 的幂)

(3) 合并问题主导 若对某常数 $\epsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 且满足

正则条件: 对某个常数 $c < 1$ 和充分大的 n 有 $a f(\frac{n}{b}) \leq c f(n)$.

则 $T(n) = \Theta(f(n))$ ($f(n)$ 的增长快于 $n^{\log_b a}$)

注意: 不适用情形: (1) 非标准式 eg: $T(n) = T(n-1) + f(n)$

(2) $f(n)$ 与 $n^{\log_b a}$ 不满足多项式差距

(3) 正则条件不满足 (如无法找到 $c < 1$)

替代方法: 手动展开 / Akra-Bazzi 处理如非均匀分割等更复杂的递归问题

$$\text{eg. } T(n) = aT(\lceil \frac{n}{b} \rceil) + \alpha n^d, \text{ 则 } T(n) = \begin{cases} O(n^d) & d > \log_b a \\ O(n^d \log n) & d = \log_b a \\ O(n^{\log_b a}) & d < \log_b a \end{cases}$$

($a \geq 0, b \geq 1, d \geq 0$)

eg. $T(n) = 9T(\frac{n}{3}) + n \quad a=4, b=3, d=1, 1 < \log_3 9 \Rightarrow T(n) = O(n^2)$

$T(n) = T(\frac{2}{3}n) + 1 \quad a=1, b=\frac{3}{2}, d=0, 0 < \log_{\frac{3}{2}} 1 \Rightarrow T(n) = O(\log n)$

$T(n) = 3T(\frac{n}{4}) + n \lg n \quad a=3, b=4, d=1, 1 < \log_4 3 \Rightarrow T(n) = O(n^{\log_4 3})$

$T(n) = 2T(\frac{n}{2}) + n \lg n \quad a=2, b=2, d=1, 1 = \log_2 2 \Rightarrow T(n) = O(n \lg^2 n)$

$T(n) = \Theta(n \lg^2 n)$

8. 算法渐近复杂度分析中常用函数

(1) 单调函数 (4) 指数函数, Taylor 展开, 极限

(2) 取整函数 (5) 对数函数, 级数 $\log(n!) = \Theta(n \log n)$

(3) 多项式函数 (6) 阶乘函数 $2 \cdot \frac{n}{2} \lg \frac{n}{2} \leq c \lg n$

$O(\frac{n}{2}) \Rightarrow f(\frac{n}{2}) \leq c f(n)$

Chapter 2 递归 (w2-4)

Re(概述) 两种效率: 时间、空间, 输入规模的函数计算算法基本操作的执行次数

一. 算法总体思想:

swap, max 等可直接用

将要求解的较大规模的问题分割为 k 个更小规模的子问题。分治法。

递归算法: 直接或间接调用自身的算法

分治法思想: 将一个难以直接解决的大问题

分解成一些规模较小的相似问题

递归函数: 用函数自身给出定义的函数

以便递归去破、分而治之

eg: 阶乘函数: $n! = \begin{cases} 1 & n=0 \text{ (边界条件)} \\ n(n-1)! & n>0 \text{ (递归方程)} \end{cases}$ 斐波那契函数 $f(n) = \begin{cases} 1 & n=0/1 \\ f(n-1)+f(n-2) & n>1 \end{cases}$

Ackerman 函数 (双递归函数)
一个函数及它的个例
由函数自身定义

$$\begin{cases} A(1,0)=2 \\ A(0,m)=1 & m>0 \\ A(n,0)=n+2 & n>2 \\ A(n,m)=A(A(n-1,m),m-1) & m,n>1 \end{cases}$$

2.1 排序问题 (全排列)

backtrack (used in solve)
int start, end
vector<vector<int>> result

2.2 整数划分个数问题

q(n,m): 最长个数 n , 不大于 m 的数

2.3 Hanio 塔问题

hanio(n-1, a, c, b);
move(a, b);
hanio(n-1, c, b, a);
问题规模减小到一定程度
即可解决

用分治法设计算法时最好使子问题规模大致相同, 平衡子问题。

$$T(n) = \begin{cases} O(1) & n=1 \\ kT(\frac{n}{m}) + f(n) & n>1 \end{cases} \Rightarrow T(n) = n^{\log_m k} + \sum_{i=0}^{\log_m n-1} k^i f(\frac{n}{m^i})$$

该问题可分解为 k 个规模相同子问题, 每个子问题规模为 $\frac{n}{m}$, 为递归问题, 即子问题不公共子问题

若认为 $T(n)$ 足够光滑, 当 $m^i \leq n < m^{i+1}$ 时, $T(m^i) \leq T(n) < T(m^{i+1})$

eg: 2.4 二分搜索技术 $O(\log n)$ while (r>l) { int m=(l+r)/2; if(x==a[m]) return m; if(x<a[m]) r=m-1; else l=m+1; }

2.5 大整数的乘法 $O(n \log^2 n)$ $XY = (a+b)(c+d) = ac + (ad+bc) + bd$

2.6 棋盘覆盖 $T(k) = \begin{cases} O(1) & k=0 \\ 4T(k-1) + O(1) & k>0 \end{cases} T(n) = O(4^k)$

2.7 合并排序 $T(n) = 2T(\frac{n}{2}) + O(n)$ 最好/平均: $O(n \log n)$ 最坏: $O(n^2)$

2.8 快速排序 $T(n) = 2T(\frac{n}{2}) + n$ 最好/平均: $O(n \log n)$ 最坏: $O(n^2)$

2.9 线性时间选择 排序 + 选择, $O(n \log n)$ Select 选择, $O(n)$

2.10 最近点对问题 $T(n) = \begin{cases} O(1) & n \leq 4 \\ 2T(\frac{n}{2}) + O(n) & n > 4 \end{cases} O(n \log n)$

2.11 循环赛日程表 $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow O(n^2)$

减少子问题个数 / 减少递归内部计算量

2.9 线性时间选择 给定线性序集 n 个元素和一个整数 k ($1 \leq k \leq n$), 找出第 k 小的元素

```
Type Select (Type a[], int p, int r, int k) {
    if (r - p < 5) { return a[p + k - 1]; }
    for (int i = 0; i <= (r - p - 4) / 5; i++)
        Type x = Select(a, p, p + (r - p - 4) / 5, (r - p - 4) / 10);
    int i = Partition(a, p, r, x), j = i - p + 1;
    if (k <= j) return Select(a, p, i, k);
    else return Select(a, i + 1, r, k - j);
}
```

2.10 空间最近点对 $T(n) = \begin{cases} O(1) & n \leq 4 \\ 2T(\frac{n}{2}) + O(n) & n > 4 \end{cases} \Rightarrow T(n) = O(n \log n)$

and $x = \text{mid}$ $d = \min\{s_1, \min, s_2, \min\}$. 只需要 p, q 且 $d(p, q) < d$

1)  2) $p, q \in x = \text{mid}$ 且 $d(p, q) < d$

$\left(\frac{d}{2}\right)^2 + \left(\frac{d}{2}\right)^2 = \frac{\sqrt{2}}{2} d^2 < d^2$ 故最多6个点 (鸽巢原理)

2.11 循环赛日程表

√	0
0	√

$T(n) = 2T(\frac{n}{2}) + O(n^2)$
 $\Rightarrow T(n) = O(n^2)$

Chapter 3 动态规划 (W5-8)

一. 算法总体思想:

将待求解问题分解为子问题, 但子问题 不互相独立; 若用分治法子问题会被重复计算
步骤: ① 找出最优解性质, 刻画结构特征 子问题是几个数往往为多项式级别

② 递归地定义最优值

③ 自底向上计算出最优值

④ 根据计算最优值时得到的信息构造最优解

eg: 斐波那契数列用备忘录数组避免重复计算
(自顶向下)

3.1 矩阵连乘问题

给定 n 个矩阵 $\{A_1, \dots, A_n\}$, 其中 A_i 与 A_{i+1} 可乘, $i=1, 2, \dots, n-1$. 考虑这 n 个矩阵的连乘积 $A_1 A_2 \dots A_n$. 由于矩阵乘法满足结合律, 故可以有多种加括号确定计算次序的方式. 问如何计算使得总乘法次数最少.

穷举法 $P(n) = \begin{cases} 1, & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k), & n>1 \end{cases} \Rightarrow P(n) = n! \left(\frac{n-1}{2}\right)$ 复杂度太高

动态规划: $A[i:j] = A_i A_{i+1} \dots A_j$ 可视为 $A[i:k] + A[k+1:j] + (A[i:k] \text{ 与 } A[k+1:j] \text{ 相乘})$ 的次序

最优子结构性质: 最优解包含着其子问题的最优解 (即 全局最优包含局部最优), 适合 DP

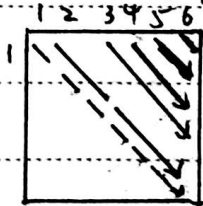
建立递归关系: 设 $A[i:j]$ 需要的最小乘法次数为 $m[i:j]$, 原问题最优值为 $m[1:n]$.

用 $s[i:j]$ 记录 $m[i:j]$ 对应的分割点 k . 设 A_i 为 $p_{i-1} \times p_i$.

当 $i=j$ 时 $A[i:j]$ 就是 A_i , $m[i:i] = 0$;

当 $i < j$ 时, $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1} p_k p_j$, 变化 k 取 $m[i,j]$ 最小值

各子问题出现 $\binom{n}{2} + n = n^2$ 次, 大量重复; 适合 DP.



由 $n-1$ 链 $\rightarrow n$ 链计算

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
for i=1 to n
```

$m[i][i]=0 \rightarrow m[i][i]$ 先置零

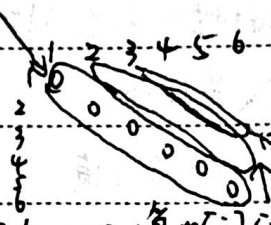
```
for r=2 to n
```

```
for i=1 to n-r+1
```

$j=i+r-1$

$m[i][j]=\infty$

← 第一次对 $i=1, \dots, n-1$ 算 $m[i][i+1]$ 第二次对 $i=1, \dots, n-2$ 算 $m[i][i+2]$



计算长度为 r 的矩阵链的最优解

for $k=i$ to $j-1 \rightarrow k$ 依次取值, 寻找使 $m[i][j]$ 最小的切割方案

$$q = m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]$$

if ($q < m[i][j]$)

$m[i][j] = q$

$s[i][j] = k$

} 时间复杂度 $O(n^3)$ 空间复杂度 $O(n^2)$

3.2 最长公共子序列

$X = \{x_1, \dots, x_m\}$, $Y = \{y_1, \dots, y_n\}$. 公共子序列 $Z = \{z_1, \dots, z_k\}$. (1) $\begin{cases} z_m \neq y_n \\ z_k \neq x_m \end{cases} \rightarrow X, Y$

(1) $x_m = y_n$, 则 z_k 是 x_{m-1} 与 y_{n-1} 公共子序列; (2) $\begin{cases} x_m \neq y_n \\ z_k \neq x_m \end{cases}$, z 是 x_m 与 y 公共子序列

$$c[i][j] = \begin{cases} 0 & i=0 \text{ 或 } j=0 \\ c[i-1][j-1] + 1 & i, j > 0, x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & x_i \neq y_j, i, j > 0 \end{cases}$$

3.5 图像压缩

[题目描述] 图像的变位压缩存储格式将所给像素点序列 $\{p_1, p_2, \dots, p_n\}$, $0 \leq p_i \leq 255$ 分成 m 个连续段 $s_1, \dots, s_m$. 第 i 个像素段 s_i ($1 \leq i \leq m$) 中有 $l[i]$ 个像素, 且该段每个像素都只用 $b[i]$ 位表示, $b[i] \leq 8$ 用 3 位表示 $b[i]$, $l[i] \in [1, 255]$, 至少用 8 位. 第 i 个像素段 s_i 的总长度为 $l[i] \cdot b[i] + 8 + 1$.

[要求] 确定像素序列 $\{p_1, \dots, p_n\}$ 的最优分段, 使得依此分段所需的存储空间最少.

总长度 $\sum_{i=1}^m (l[i] \cdot b[i] + 11) \cdot m$ 固定开销 $(8+3) \times$ 分段点数 vs 分段节约的开销

[题目分析] 设 $l[i], b[i], 1 \leq i \leq m$ 是 $\{p_1, \dots, p_n\}$ 的最优分段. 显然 $\forall i \in [1, m]$, $l[i], b[i]$ 都是对应子序列的最优分段, 满足最优子结构.

$$s[i] = \min_{1 \leq k \leq \min(l[i], n)} \{s[i-k] + k \cdot b_{\max(i-k+1, i)}\} + 1$$

$a \sim p_n$ 有 $k+1$ 种可能

其中 $b_{\max}(i, j) = \lceil \log(\max_{i \leq k \leq j} \{p_k\} + 1) \rceil$ 为 p_i 到 p_j 中, 最大的值需要的比特位数.

[举例] 求 4, 6, 5, 7, 129, 138, 1 的最优分段

$$\textcircled{1} s[1] = 3 + 11 = 14 \quad l[1] = 1, b[1] = 3$$

$$\textcircled{2} s[2] = \min \begin{cases} s[0] + 2 \times 3 + 11 \\ s[1] + 3 + 11 \end{cases} = \min\{17, 28\} = 17$$

以此类推, 可计算出 s_n

[代码] void Compress (int n, int p[], int s[], int l[], int b[]) {

int Lmax = 256, header = 11;

s[0] = 0;

for (int i = 1; i <= n; i++) {

b[i] = length(p[i]); // 计算像素点 p 需要的存储位数

int bmax = b[i];

s[i] = s[i-1] + bmax; // 下面 j 从 1 开始

l[i] = 1;

for (int j = 2; j <= i && j <= Lmax; j++) { // s[i] = min { s[i-j] + (smax(i-j+1, i) * bmax(i-j+1, i) + 1) }

if (bmax < b[i-j+1]) bmax = b[i-j+1];

if (s[i] > s[i-j] + j * bmax) {

s[i] = s[i-j] + j * bmax;

l[i] = j;

}

s[i] += header;

}

3.6 流水作业调度

[问题描述] n 个作业 $\{1, \dots, n\}$ 由 2 台机器 M_1, M_2 组成的流水线上完成加工。先 M_1 后 M_2 。要求确定 n 个作业的最优加工顺序, 使得从第一个作业在 M_1 上加工到最后一个作业在 M_2 上加工完成所需的时间最少。

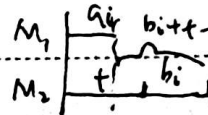
[求解] $T(s, t)$: M_1 开始加工 S 作业 i 时 M_2 要等待 T 后可利用

1° $a_i > t$



不用等 $T(s, t) = \{a_i + T(s - \{i\}, b_i)\}$
 $i \in s$

2° $a_i < t$



$T(s, t) = \{a_i + T(s - \{i\}, b_i + \max\{t - a_i, 0\})\}$
 $i \in s$

2) $T(s, t) = \{a_i + T(s - \{i\}, b_i + \max\{t - a_i, 0\})\}$ $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$

由于子问题个数为 N 的子问题个数 $2^n - 1$ 指数级复杂度, 故引入 Johnson 不等式: 最优

① 展开, $T(s, t) = a_i + T(s - \{i\}, b_i + \max\{t - a_i, 0\}) = a_i + a_j + T(s - \{i, j\}, t_{ij})$.

其中 $t_{ij} = b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} = b_i + b_j - a_i - a_j + \max\{t, a_i + a_j - b_i, a_j\}$

(1) 令 $N_1 = \{i \mid a_i < b_i\}$, $N_2 = \{i \mid a_i > b_i\}$;

(2) 将 N_1 中作业按 a_i 的非减序排序; 将 N_2 中作业按 b_i 的非增序排序

(3) N_1 中作业接 N_2 中作业构成满足 Johnson 法则的最优调度。

3.7 0-1 背包问题

[题目描述] 给定 n 种物品和一背包。物品 i 重量为 w_i ，其价值为 v_i ，背包容量为 C 。

问怎么装放入背包的物品，使得装入背包中物品总价值最大？设 x_i 表示是否将物品 i 装入背包

0-1 背包问题是一个特殊的整数规划问题。求 $\max \sum_{i=1}^n v_i x_i$ ，满足 $\begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases}$

[求解] 最优子结构。设 $m(i, j)$ 是背包容量为 j 、可选择物品为 $i, i+1, \dots, n$ 时 0-1 背包最优值。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\}, & j \geq w_i \\ m(i+1, j), & 0 \leq j < w_i \end{cases}$$

能装下，装/不装取 $\max$
装不下，跳过该物品

$$m(n, j) = \begin{cases} v_n, & j \geq w_n \\ 0, & 0 \leq j < w_n \end{cases}$$

7.8 最优二叉搜索树

时间复杂度 $O(n^2)$

Chapter 4 贪心算法

只做出局部最优选择, 在单源最短路径、最小生成树等问题恰好为整体最优解。

4.1 活动安排问题

问题描述: n 个活动的集合 $E = \{1, 2, \dots, n\}$, 每个活动要求使用同一资源, 在同一时间内只有一个活动能使用这一资源。

按结束时间作非减序排列

若被检查活动 i : 开始时间 $s_i <$ 最近选择活动 j 的结束时间 f_j , 则不选择; 否则选择, 加入 P (最优解一定包含结束时间是早的活动) 44

贪心性质, 数学归纳法验证

4.2 贪心算法的基本要素

1. 贪心选择性质: 所求问题的整体最优解可通过一系列局部最优的选择 (即贪心选择) 达到。

动态规划: 自底向上 贪心算法: 自顶向下

2. 最优子结构性质

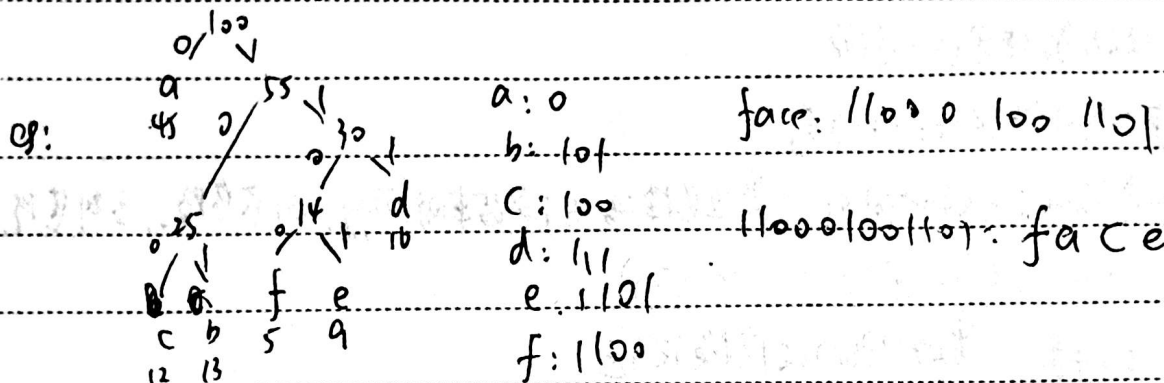
eg, 0-1 背包应用 DP, 普通可分割背包用贪心。

适用于组合优化问题

4.3 最优装载

问题描述: 有一批集装箱(n 个)要装上一艘载重量为 c 的轮船。其中集装箱 i 重量为 w_i 。要求体积不受限情况下将尽可能多的集装箱装上轮船。

按单位重量从大到小排序



4.4 哈夫曼编码 $O(n \log n)$

前缀码: 对每一个字符规定一个0,1串且互不重叠 平均码长 $\sum f(c) d(c)$ 最短

Re: 贪心选择, 整体最优 $\leftarrow$ 局部最优

最优子结构

4.5 单源最短路径 $O(n^2)$

特殊路径: 只经过顶点集 S 里有的点

4.6 最小生成树

~~MST~~ MST性质: (贪心)

Prim: 按权值从小到大 $O(n^2)$ 稠密

Kruskal: 全局权值也 稀疏 (不成环啊)

4.7 多机调度问题 NP完全

最长处理时间作业优先调度算法 $O(n \log n)$

Chapter 5 回溯法 深度优先 DFS

找解集/满足条件的条件的最佳解

解空间: ① 解向量 $(x_1, \dots, x_n)$

② 显约束: x_i 取值限定

③ 隐约束: 对不同分量解向量^{施加}约束

④ 解空间: 满足显约束的解向量

扩展结点: 一个正在产生儿子的结点

活结点: 一个自身已生成但其儿子还没有全部生成的节点

死结点: 一个所有儿子都已生成的结点

{ 约束函数: 在扩展结点处剪去不满足约束的子树 可行解

也可非递归: 迭代

限界函数: 剪去得不到最优解的子树 最优解

基本思想: (1) 定义解空间 (2) 确定易于搜索的解空间结构 (3) DFS + 剪枝

eg1. 0-1 背包 子集树 — 装载; 符号三角形; 0-1 背包; 最大团问题 2^n

eg2. 旅行售货员问题是 排列树 — 批处理作业; n 皇后; 旅行; 图着色; 电路板 $n!$

void backtrack(int t){

if (t > n) output(x); // 遇到叶子节点 输出结果

else // 遍历 t 的子节点 递归 best 数组

for (int i = f(t); i <= g(t); i++) {

x[t] = h(i);

if (constraint(t) && bound(t)) backtrack(t+1);

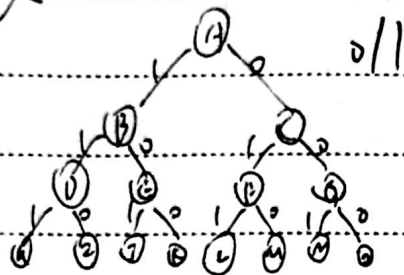
legal(t)

}

}

2^n 子集树, n 元组 $x_i = (0/1, 0/1, \dots, 0/1) \in (2^n)$

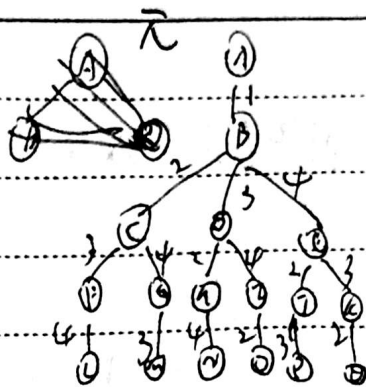
$n!$ 排列树: $x_i = (\dots) \in (n!)$



子集树

$O(2^n)$

```
void backtrack(int t) {
    if (t > n) output(x); // 输出
    else
        for (int i = 0; i <= 1; i++) {
            x[t] = i;
            if (legal(t)) backtrack(t+1);
            // constraint + bound
        }
}
```



排列树 $O(n!)$

```
void backtrack(int t) {
    if (t == n) output(x);
    else
        for (int i = t; i <= n; i++) {
            swap(x[t], x[i]);
            if (legal(t)) {
                backtrack(t+1);
            }
            swap(x[t], x[i]);
        }
}
```

5.2 装载问题 $O(2^n)$ 子集树

$$\sum_{i=1}^n w_i \leq C_1 + C_2$$

可行性约束 constraint: $\sum_{i=1}^n w_i x_i \leq C_i$

最优方案: 1) 先装第一艘轮船

2) 装第二艘, 当前载重量 cur + 剩余重量 r

3) 将剩余集装箱装上第二艘轮船

$\leq$ 当前最优载重量 $bestw$

5.3 批处理作业调度 排列树

解空间 $(x_1, \dots, x_n)$

5.4 符号三角问题 子集树

解向量 n 元组 $(x_1, \dots, x_n)$ 约束: $x_i \in \{0, 1\}, 1 \leq i \leq n$

可行性约束: "+" "-" 均不超过 $\frac{n+1}{4}$, 且给定 $\frac{n+1}{2}$ 不为奇数 $O(n)$

每种情况 $O(2^n)$ 结点 总时间 $O(n2^n)$

$$p[i][j] = p[i-1][j] \oplus p[i-1][j+1] \quad \begin{matrix} 0 \rightarrow + \\ 1 \rightarrow - \end{matrix}$$

5.5 n 皇后问题 排列树 $n!$

$S = \{1, \dots, n\}, 1 \leq i \leq n, x_i \neq x_j (1 \leq i, j \leq n)$

约束: $i \neq j$ 时 $|x_i - x_j| \neq |i - j|$
无解

5.6 0-1背包 子集树

约束: $W_i \leq C, \sum_{i=1}^n u_i x_i \leq W, 0 \leq x_i \leq 1 \Rightarrow O(2^n) = O(2^n)$

限界: $C_p + r_p > best_u, O(n 2^n)$

5.7 最大团问题 最大完全子图 子集树

完全图: $\forall$ 两点, 均有一条边相连

完全子图: $\forall$ 两点均有一条边相连的子图

团: 最大的完全子图

约束: OK

限界: $C_n + r_n > best_n$

with clique: Backtrack (int t)

if (t > n) {

for (int j = 1; j <= n; j++)

best x[j] = x[j];

best cn = cn; return j;

int ok = 1;

for (int j = 1; j < i; j++)

if (x[j] & a[j][i] & !x[i])

{ ok = 0; break; }

if (!ok) {

x[i] = 1; cntd;

backtrack (i+1);

x[i] = 0; cntd--;

if (cn + n = i > best_n) {

x[i] = 0;

Backtrack (i+1);

5.8 图的m着色问题 排列树

$a[i][j] = \begin{cases} 1 & \text{if } (i,j) \in E \text{ 叶子节点数 } m^1 \\ 0 & \text{others 每个内部节点孩子数 } m \end{cases}$

解空间 $(x_1, \dots, x_n)$ 表示顶点 i 的着色 $x[i]$

约束 $\forall i \neq j, \text{若 } a[i][j] = 1, \text{则 } x[i] \neq x[j]$

5.9 旅行售货员问题 排列树 $O(n!)$

解形式 $(x_1, \dots, x_n)$

约束: $a[i][j] \neq 0$ 的边

限界: $cc < best_c$ ($best_c$ 初值为 ∞) cc : 当前已选长度

5.10 图排列问题 排列树 $O(n!)$

$x_t = \max_{1 \leq j \leq n} \{x_j + 2\sqrt{r_j}\}$

5.11 连续邮资问题

(1) $x[k]$ 产生时间

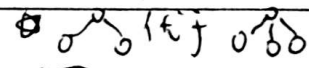
(2) 满足是约束 $x[k]$ 个数

(3) 计算 constraint 时间

(4) bound 时间

(5) 满足 (3) (4) 的 $x[k]$ 时间

重排内理: 让取值最少的 $x[i]$ 优先



每个节点只有一块成为根节点机会 Chapter 6 分支限界法 BFS 选一个节点

队列式 (FIFO): 按照先进先出原则选下一节点作为 最大收益优先 最大堆

优先队列式: 根据优先队列确定优先权最高节点作为根节点 最大堆/最小堆

(1) FIFO 队列 取头、生子、过滤、入尾

$\{\} \rightarrow \{A\} \rightarrow \{B, C\} \rightarrow \{C, D, E\} \rightarrow \{C, E\} \rightarrow \{E, F, G\} \rightarrow \{F, G, J, K\} \rightarrow \{E, G, K\}$
 $\rightarrow \{G, K, L, M\} \rightarrow \{K, L, M, N, O\} \rightarrow \{\}$ 先进先出, 也是也筛选(过滤)

(2) 优先队列 价值大者优先

步骤: (1) 定义解空间

2) 确定用于搜索的解空间结构 (二叉树)

3) 构造数据结构

4) 限界, 约束剪枝

5) 确定活节点组织方法

6) 确定劣化节点识别方法

6.2 单源最短路径问题 (Dijkstra)

剪枝: 一个节点下界不小于当前找到的最短路径长.

6.3 装载问题

6.4 0-1 背包问题

6.5 最大团问题

6.6 旅行售货员问题

6.7 批处理作业调度问题

Re (Chapter 6) 分支限界法

1. 与回溯法的区别:

分支限界法 回溯法

求解目标: 最优解 vs 所有解

搜索方式: BFS vs DFS

扩展方式: 一次性所有儿子 vs 逐个产生

2. 两种分支限界法

(1) 队列式 (FIFO) 分支限界法

(2) 优先队列式分支限界法

Re (Chapter 8) 线性规划

1. 理解线性规划问题的模型和解的概念: 决策变量、目标函数、约束条件

2. 掌握线性规划基本定理

图解直观, 但无法证明/高维无法
求解/无最优值 处理

3. 若有最优解, 则必有一基本可行最优解

4. 单纯形法: (1) 入基变量 (2) 出基变量 (3) 转轴变换 (4) 判断最优

5. 掌握一般线性规划问题的处理: 大M法、两阶段法

三要素: 决策变量、目标函数、约束条件

可行区域: 所有可行解构成的集合

线性规划基本定理: 可行满足称为基本可行解

单纯形法: (1) 每次, 目标函数的值都增加 (2) 不超过 m/n 次就能解

计算步骤: (1) 入基, 找出目标函数所有 $c_j < 0$

(2) 出基, 构造序列, 若所有 $a_{ie} \leq 0$ 则无界; 否则 $\theta = \min_{a_{ie} > 0} \{ \frac{b_i}{a_{ie}} \}$

将一般问题转为标准形

(1) 若求 $\min$, 可令 $\min z = \sum C_j X_j$, 令 $\max z' = -z = -\sum C_j X_j$.

(2) 引入松弛变量, 利用松弛变量非负性将不等式转为等式

eg: $X_1 + 2X_3 \leq 18 \Rightarrow X_1 + 2X_3 + y_1 = 18$

$z_1 + X_1 + 2X_3 + y_1 = 18$

$2X_2 - 7X_4 \leq 0 \Rightarrow 2X_2 - 7X_4 + y_2 = 0$

$z_2 + 2X_2 - 7X_4 + y_2 = 0$

$X_1 + X_2 + X_3 + X_4 = 9 \Rightarrow$ 不变

$\Rightarrow z_3 + X_1 + X_2 + X_3 + X_4 = 9$

$X_2 - X_3 + 2X_4 \geq 1 \Rightarrow X_2 - X_3 + 2X_4 - y_3 = 1$

$z_4 + X_2 - X_3 + 2X_4 - y_3 = 1$

人工变量法 大M法/两阶段法

eg: $\min z = X_1 + X_2 + X_3$

Step 1: $\min \rightarrow \max(-z)$

s.t. $\begin{cases} 2X_1 + 7.5X_2 + 3X_3 \geq 10000 \\ 20X_1 + 5X_2 + 10X_3 \geq 30000 \end{cases}$

Step 2: 约束 - 松弛变量 + 人工变量

$X_1, X_2, X_3 \geq 0$

Step 3: 第 1 阶段求辅助变量 $\min(y_1, y_2)$

Step 4: 第二阶段单纯形

1. $\min \rightarrow \max$ 修改目标, $\max z = -z = -X_1 - X_2 - X_3$

2. 松弛变量 $\begin{cases} 2X_1 + 7.5X_2 + 3X_3 - y_1 = 10000 \\ 20X_1 + 5X_2 + 10X_3 - y_2 = 30000 \\ X_1, X_2, X_3, y_1, y_2 \geq 0 \end{cases}$

3. 人工变量 $\begin{cases} z_1 + 2X_1 + 7.5X_2 + 3X_3 - y_1 = 10000 \\ z_2 + 20X_1 + 5X_2 + 10X_3 - y_2 = 30000 \\ X_1, X_2, X_3, y_1, y_2, z_1, z_2 \geq 0 \end{cases}$

4. 第一阶段目标函数 $\min(z_1 + z_2) \Rightarrow z' = \max(-z_1 - z_2) = -2X_1 - 12.5X_2 - 13X_3 - y_1 - y_2 - 40000$

X_1	X_2	X_3	y_1	y_2	z_1	z_2	值
z'	-40000	-2	-12.5	-13	-1	-1	
z_1	10000	2	7.5	3	-1	0	-7000
z_2	30000	20	5	10	0	-1	7000
z	0	-1	-1	-1	0	0	1500
							-1500

约束标准型线性规划问题的单纯形算法

$$\max Z = \sum_{j=1}^n c_j x_j$$

$$s.t. \begin{cases} \sum_{j=1}^n a_{ij} x_j = b_i \quad (i=1, 2, \dots, m) \\ x_j \geq 0, j=1, 2, \dots, n \end{cases}$$

基解: $x = (7, 0, 0, 12, 0, 10)$

eg: $\max Z = -x_2 + 3x_3 - 2x_5$

$$\begin{cases} x_1 + 3x_2 - x_3 + 2x_5 = 7 \\ x_4 - 2x_2 + 4x_3 = 12 \\ x_6 - 4x_2 + 3x_3 + 8x_5 = 10 \end{cases}$$

$x_i \geq 0, i=1, 2, 3, 4, 5, 6$

	x_2	x_3	x_5
Z	0	-1	-2
x_1	7	3	-1
x_4	12	-2	4
x_6	10	-4	3

$\min \{12, 10, \frac{10}{3}\} = 3$

$$x_3 - \frac{1}{2}x_2 + \frac{1}{4}x_4 = 3$$

消去 x_3

转轴变换:

$$x_1 + \frac{5}{2}x_2 + \frac{1}{4}x_4 + 2x_5 = 10$$

$$x_6 - \frac{5}{2}x_2 - \frac{3}{4}x_4 + 8x_5 = 1$$

$$Z = 9 + \frac{1}{2}x_2 - \frac{3}{4}x_4 - 2x_5$$

	x_2	x_4	x_5
Z	9	$\frac{1}{2}$	$-\frac{3}{4}$
x_1	10	$\frac{5}{2}$	$\frac{1}{4}$
x_3	3	$-\frac{1}{2}$	$\frac{1}{4}$
x_6	1	$-\frac{5}{2}$	$\frac{3}{4}$

~~$$x_2 = \frac{1}{2}x_4 + \frac{1}{4}x_5 - 6$$~~

~~$$x_1 + \frac{5}{2}(\frac{1}{2}x_4 + \frac{1}{4}x_5 - 6) + \frac{1}{4}x_4 + 2x_5 = 10$$~~

~~$$x_1 + \frac{7}{8}x_4 + \frac{5}{8}x_5 + 2x_5 = 25$$~~

~~$$\frac{5}{2}x_1 + x_2 + \frac{1}{10}x_4 + \frac{4}{5}x_5 = 4$$~~

~~$$Z = 9 + \frac{1}{2}(\frac{1}{2}x_4 + \frac{1}{4}x_5 - 6) - \frac{3}{4}x_4 - 2x_5$$~~

~~$$x_2 = 4 - \frac{3}{5}x_1 - \frac{1}{10}x_4 - \frac{4}{5}x_5$$~~

~~$$Z = 9 + 2 - \frac{1}{5}x_1 - \frac{1}{20}x_4 - \frac{3}{5}x_5 - \frac{3}{4}x_4 - 2x_5$$~~

~~$$= 11 - \frac{1}{5}x_1 - \frac{4}{5}x_4 - \frac{13}{5}x_5$$~~

~~$$x_3 + \frac{1}{4}x_4 - 3 = 2 - \frac{1}{5}x_1 - \frac{1}{20}x_4 - \frac{2}{5}x_5$$~~

~~$$x_5 + \frac{3}{10}x_4 + \frac{1}{5}x_1 + \frac{2}{5}x_5 = 5$$~~

~~$$x_6 + 8x_5 - \frac{5}{2}(4 - \frac{3}{5}x_1 - \frac{1}{10}x_4 - \frac{4}{5}x_5) - \frac{3}{4}x_4 = 1$$~~

~~$$x_6 + 8x_5 - 10 + 15x_1 + 2x_4 + 2x_5 = 1$$~~

两阶段法例题:

$$z = \max (3x_1 + x_2 - 2x_3)$$

注: 第一阶段 $\min (x_5 + x_6 + x_7)$ $x_j \geq 0, j \in \{1, 2, 3, 4, 5, 6, 7\}$

$$\begin{cases} 2x_1 - x_2 + x_3 = 4 \\ x_1 + x_2 + x_3 = 6 \\ x_1 + x_4 = 2 \\ 3x_1 + 2x_3 = 10 \end{cases} \xrightarrow{\text{引入松弛变量}} \begin{cases} 2x_1 - x_2 + x_3 + x_5 = 4 \\ x_1 + x_2 + x_3 + x_6 = 6 \\ x_1 + x_4 = 2 \\ 3x_1 + 2x_3 + x_7 = 10 \end{cases}$$

初始单纯形表

值	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_5 4	2	-1	1		1		
x_6 6	1	1	1			1	
x_4 2	1			1			
x_7 10	3		2				1
目标 20	6	0	4				

第一次迭代

值	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_5 0	0	-1	1		1		
x_6 4	0	2	0		1	1	
x_4 2	0	1		1			
x_7 4	0	2	0		1	1	1
目标 8	0	4	0		0	0	0

第二次迭代

值	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_5 0	0	-1	1		1		
x_6 4	0	2	0		1	1	
x_4 2	0	1		1			
x_7 4	0	2	0		1	1	1
目标 8	0	4	0		0	0	0

可行基可分解 $(x_1, x_2, x_3, x_4) = (2, 2, 0, 0)$
 x_5 变量 $(x_5, x_6, x_7) = (0, 0, 0)$

$$z = 3x_1 + x_2 - 2x_3 = 3(2 - x_4) + 2 - \frac{1}{2}x_6 - 2(2 + \frac{3}{2}x_4)$$

$$z_{\max} = 4 \quad (x_1, x_2, x_3, x_4) = (2, 2, 0, 0)$$

让 x_1 入基, 那么就要把 x_1 解出来, $x_1 = 2 - x_4 \Rightarrow 2 = x_1 + x_4$

$$(4) \quad 2x_1 - x_2 + x_3 + x_5 = 4 \Rightarrow 2(2 - x_4) - x_2 + x_3 + x_5 = 4 \Rightarrow -x_2 + x_3 - 2x_4 + x_5 = 0$$

$$(4) \quad x_1 + x_2 + x_3 + x_6 = 6 \Rightarrow (2 - x_4) + x_2 + x_3 + x_6 = 6 \Rightarrow x_2 + x_3 - x_4 + x_6 = 4$$

$$(4) \quad 3x_1 + 2x_2 + x_7 = 10 \Rightarrow 3(2 - x_4) + 2x_2 + x_7 = 10 \Rightarrow 2x_2 - 3x_4 + x_7 = 4$$

$$(1) \quad 6x_1 + 4x_3 = 20 \Rightarrow 6(2 - x_4) + 4x_3 = 20 \Rightarrow 4x_3 - 6x_4 = 8$$

$$0 = -x_2 + x_3 - 2x_4 + x_5$$

让 x_3 入基, 那么就是 $-x_2 + x_3 - 2x_4 + x_5 = 0 \Rightarrow x_3 = x_2 + 2x_4 - x_5$

$$(4) \quad 4 = x_2 + x_3 - x_4 + x_6 \Rightarrow x_2 + x_2 + 2x_4 - x_5 - x_4 + x_6 = 4 \Rightarrow 2x_2 + x_4 - x_5 + x_6 = 4$$

$$(4) \quad x_1 = 2 - x_4, \text{ 不过; } (4) \quad 2x_2 - 3x_4 + x_7 = 4 \Rightarrow 2x_2 + x_4 - 2x_5 - x_6 + x_7 = 4$$

$$(1) \quad 4x_3 - 6x_4 = 8 \Rightarrow 4x_2 + 8x_4 - 4x_5 - 6x_4 = 8 \Rightarrow 4x_2 + 2x_4 - 4x_5 = 8$$

让 x_2 入基, 也即 $2x_2 + x_4 - x_5 + x_6 = 4 \Rightarrow x_2 = 2 - \frac{1}{2}x_4 + \frac{1}{2}x_5 - \frac{1}{2}x_6$

$$\text{先解决 } x_2 (\lambda = 1/2) \quad 2 = x_2 + \frac{1}{2}x_4 - \frac{1}{2}x_5 + \frac{1}{2}x_6$$

$$(1) \quad -2 + \frac{1}{2}x_4 - \frac{1}{2}x_5 + \frac{1}{2}x_6 + x_3 - 2x_4 + x_5 = 0 \Rightarrow x_3 - \frac{3}{2}x_4 + \frac{1}{2}x_5 + \frac{1}{2}x_6 = 2$$

$$(4) \quad \text{不过; } (4) \quad 4 - x_4 + x_5 - x_6 + x_4 - 2x_5 + x_7 = 4 \Rightarrow -x_5 - x_6 + x_7 = 0$$

$$(4) \quad \text{同样; } 4x_2 + 2x_4 - 4x_5 = 8 \Rightarrow 4(2 - \frac{1}{2}x_4 + \frac{1}{2}x_5 - \frac{1}{2}x_6) + 2x_4 - 4x_5 = 8$$

$$\Rightarrow 8 - 2x_4 + 2x_5 - 2x_6 + 2x_4 - 4x_5 = 8 \Rightarrow -2x_5 - 2x_6 = 0$$

算法第一次作业

1. $\lfloor \sqrt{n} \rfloor$ 的算法:

```
def floor_sqrt(n):
```

```
    if n < 0:
```

```
        print('负数的算术平方根不存在')
```

```
    if n == 0:
```

```
        return 0
```

```
    low = 0
```

```
    high = n
```

```
    ans = 0 # 存储最大的小于等于 sqrt(n) 的整数
```

```
    while low <= high: # 二分法, 折半查找
```

```
        mid = (low + high) // 2
```

```
        if mid * mid == n:
```

```
            return mid
```

```
        elif mid * mid < n:
```

```
            ans = mid
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return ans
```

$\sqrt{25} = 5$

$n = 25, \text{tar} = 5$

tar = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25

i = 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

i = 1 2 3

i = 2 3

2. 1-b (1) $f(n) = 2 \log n$. $\exists C_1 = 0.01, C_2 = 100, \forall n \geq 1000000, C_1 g(n) \leq f(n) \leq C_2 g(n)$

$\therefore f(n) = \Theta(g(n))$

(2) $\exists C = 1, \forall n \geq 0, f(n) \leq C g(n) \therefore f(n) = O(g(n))$

(3) $\exists C = 0.1, \forall n \geq 100, C g(n) \leq f(n) \therefore f(n) = \Omega(g(n))$

$$(4) \exists c=1, \forall n \geq 0, c g(n) \leq f(n) \leq f(n) = \Omega(g(n))$$

$$(5) \exists C_1=0.0001, C_2=6000, \forall n \geq 0, C_1 f(n) \leq g(n) \leq C_2 f(n) \therefore f(n) = \Theta(g(n))$$

$$(6) \lim_{n \rightarrow \infty} \frac{(\log n)^2}{\log n} = \lim_{n \rightarrow \infty} \log n = \infty \therefore f(n) = \Omega(g(n))$$

$$(7) \lim_{n \rightarrow \infty} \frac{2^n}{(\log n)^2} = +\infty \Rightarrow f(n) = \Omega(g(n))$$

$$(8) \lim_{n \rightarrow \infty} \frac{2^n}{3^n} = \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0 \Rightarrow f(n) = o(g(n))$$

$$1-7 \text{ pf: } n! = o(n^n)$$

$$\frac{n!}{n^n} = \frac{1 \times 2 \times \dots \times n}{n \times n \times \dots \times n} = \frac{1}{n} \cdot \frac{2}{n} \cdot \dots \cdot \frac{n}{n}, \frac{1}{n}, \dots, \frac{n-1}{n} \in (0,1) \Rightarrow 0 < \frac{n!}{n^n} \leq \frac{1}{n}$$

$$\text{当 } n \rightarrow +\infty, \text{ 由 } \lim_{n \rightarrow \infty} 0 = \lim_{n \rightarrow \infty} \frac{1}{n} = 0 \text{ 及夹逼定理可知 } \lim_{n \rightarrow \infty} \frac{n!}{n^n} = 0 \text{ 即 } n! = o(n^n)$$

($i \geq n$ 时 while 不执行)

$$3. (1) \text{ 设“循环执行” } k \text{ 次, } 2^k \geq n > 2^{k-1} \Rightarrow k = \log_2 n \text{ 复杂度为 } O(\log n)$$

$$(2) T(n) = 2T\left(\frac{n}{2}\right) + n. \text{ 由主定理 } a=b=2, f(n)=n. \text{ 比较 } f(n) \text{ 和 } n^{\log_2 2} = n. \\ f(n) = \Theta(n^{\log_2 2}) \Rightarrow \text{时间复杂度为 } \Theta(n \log n)$$

$$(3) \text{ “循环执行” } n \text{ 次, 中间“循环执行” } \frac{n}{2} \text{ 次, } k \text{ 次执行 } n \text{ 次, 复杂度 } O(n^3)$$

$$(4) \text{ 同 (3)}$$

分析题

2-7 由 $P(x) = \dots = P(x-d) = 0$ 得 $P(x) = \prod_{i=1}^d (x-n_i) = \prod_{i=1}^{\frac{d}{2}} (x-n_i) \cdot \prod_{i=\frac{d}{2}+1}^d (x-n_i)$

分析: 把 d 个多项式变为 2 个规模 $\frac{d}{2}$ 的多项式 $\hat{=} P_1(x)P_2(x)$

设计算 d 次多项式所需时间复杂度为 $T(d)$

$$T(d) = \begin{cases} O(1), & d=1 \\ 2T(\frac{d}{2}) + O(d \log d), & d>1 \end{cases}$$

$a=b=2, \log_2 a=1 \quad f(n) = O(d \log^2 d) \Rightarrow T(n) = O(n \log^2 n)$

2-15 n 为偶数 将 n 个选手分为 2 组 每组 $\frac{n}{2}$ 个选手即可

n 为奇数 增加一个虚拟选手, 使总数变为 $n+1$ 人, 同偶数情况安排比赛, 最后去掉虚拟选手的比赛记录即可

设有 p 个选手, $T(p) = \begin{cases} O(1), & p=2 \\ T(\frac{p}{2}) + O(p^2), & p \text{ 为大于 2 的偶数} \\ T(p+1), & p \text{ 为大于 2 的奇数} \end{cases}$ 则 $T(p) = O(p^2)$
时间复杂度为 $O(n^2)$

设计题

2-6/2-9